```
for _, pred := range predictions {
     file.WriteString(fmt.Sprintf("%v\n", pred))
}
```

Explanation:

- Writes predictions to a CSV file for external visualisation.

Clustering techniques

Example: K-Means clustering implementation

```
import "github.com/sjwhitworth/golearn/clustering"

// Create a K-Means model
kmeans := clustering.NewKMeans(3, 10) // 3 clusters, 10
iterations
kmeans.Fit(trainingData) // Train the model

// Assign test data to clusters
clusters := kmeans.Predict(testData)
```

Explanation:

- *NewKMeans(3, 10)* initialises K-Means with 3 clusters and 10 iterations.
- *Fit(trainingData)* trains the model.
- *Predict(testData)* assigns test samples to clusters.

Analysing clusters and use cases

```
// Count occurrences of each cluster
clusterCounts := make(map[int]int)
for _, cluster := range clusters {

     clusterCounts[cluster]++
}

// Print cluster distribution
fmt.Println("Cluster Distribution:", clusterCounts)
```

Use cases of K-Means clustering

- **Customer segmentation**: Group customers based on purchasing behaviour.
- **Anomaly detection**: Identify outliers in network security.
- **Image segmentation**: Group similar pixels in images.

Performance optimisation tips for GoLearn

Optimising machine learning performance in GoLearn requires efficient data handling, leveraging Go's concurrency model, and refining model evaluation techniques.

Efficient data handling with Go

Handling large datasets is key, and GoLearn benefits from Go's memory-efficient data structures with potential for further performance improvements.

Use buffered I/O for faster data loading: Instead of reading large CSV files directly, using buffered I/O improves speed and reduces memory usage.

```
import (
     "bufio"
     "os"
)

// Function to read a CSV file efficiently
func readCSV(filePath string) {
     file, err := os.Open(filePath)
     if err != nil {
       panic(err)
     }
     defer file.Close()

     scanner := bufio.NewScanner(file)
     for scanner.Scan() {
       // Process each line efficiently
       line := scanner.Text()
       _ = line // Replace with actual processing
     }

     if err := scanner.Err(); err != nil {
       panic(err)
     }
}
```

Use memory-mapped files for large datasets: Memory mapping allows large datasets to be accessed without fully loading them into RAM.

```
import (
     "os"
     "syscall"
)

// Function to map a file into memory
func mapFile(filePath string) []byte {
     file, err := os.Open(filePath)
     if err != nil {
       panic(err)
     }
     defer file.Close()

     // Get file size
     fileInfo, _ := file.Stat()
     fileSize := fileInfo.Size()

     // Memory-map the file
     data, err := syscall.Mmap(int(file.Fd()), 0,
```